AJAY.I - 71812401007 · MICROCONTROLLER AND PROGRAMMING · REVISION CODES

Experiment 1a — Addition and subtraction of two 8-bit numbers

1. Addition without carry

```
ORG 0000H
CLR C
MOV A,#18H
ADD A,#25H
MOV R0,A
HERE: SJMP HERE
END
```

2. Addition with carry

```
ORG 0000H
CLR C
MOV A,#0B2H
ADD A,#5AH
MOV R1,A
HERE: SJMP HERE
END
```

3. Subtraction without borrow

```
ORG 0000H
CLR C
MOV A,#58H
SUBB A,#1EH
MOV R2,A
HERE: SJMP HERE
END
```

4. Subtraction with borrow

```
ORG 0000H
CLR C
MOV A,#2AH
SUBB A,#45H
MOV R3,A
HERE: SJMP HERE
END
```

Experiment 1b — Multiplication and division of two 8-bit numbers

1. Multiplication without overflow

```
ORG 0000H
MOV A,#08H
MOV B,#06H
MUL AB
MOV R4,A
HERE: SJMP HERE
END
```

2. Multiplication with overflow

```
ORG 0000H
MOV A,#28H
MOV B,#14H
MUL AB
MOV R5,A
MOV R6,B
HERE: SJMP HERE
END
```

3. Division without remainder

```
ORG 0000H
MOV A,#40H
MOV B,#08H
DIV AB
MOV R7,A
HERE: SJMP HERE
END
```

4. Division with remainder

```
ORG 0000H
MOV A,#3BH
MOV B,#08H
DIV AB
MOV R0,A
MOV R1,B
HERE: SJMP HERE
END
```

Experiment 1c — Logical AND and OR

1. Logical AND

```
ORG 0000H
MOV A,#5AH
ANL A,#3CH
MOV R2,A
HERE: SJMP HERE
END
```

2. Logical OR

```
ORG 0000H
MOV A,#44H
ORL A,#13H
MOV R3,A
HERE: SJMP HERE
END
```

Experiment 1d — Logical NOT and XOR

1. Logical NOT (complement)

```
ORG 0000H
MOV A,#55H
CPL A
MOV R5,A
HERE: SJMP HERE
END
```

2. Logical XOR

```
ORG 0000H
MOV A,#5AH
XRL A,#33H
MOV R4,A
HERE: SJMP HERE
END
```

Experiment 2 — Data transfer between memory blocks

2a. Transfer four bytes from 60H to 70H

```
ORG 0000H
MOV R0,#60H
MOV R1,#70H
MOV R2,#04H
L1: MOV A,@R0
MOV @R1,A
INC R0
INC R1
DJNZ R2,L1
HERE: SJMP HERE
END
```

2b. Exchange four bytes between 50H and 80H

```
ORG 0000H
MOV R0,#50H
MOV R1,#80H
MOV R2,#04H
L1: MOV A,@R0
MOV B,A
MOV A,@R1
MOV @R0,A
MOV A,B
MOV @R1,A
INC R0
INC R1
DJNZ R2,L1
HERE: SJMP HERE
END
```

Experiment 3 — Largest and smallest numbers in an array

3a. Find the largest number

```
ORG 0000H
MOV R2, 50H
DEC R2
MOV R0, #60H
MOV A, @R0
INC R0
L1: MOV B, A
MOV A, @R0
CJNE A, 0F0H, L2
SJMP L3
L2: JNC L3
MOV A, B
```

```
L3: INC R0
DJNZ R2, L1
MOV 70H, A
HERE: SJMP HERE
END
```

3b. Find the smallest number

```
ORG 0000H
MOV R2, 50H
DEC R2
MOV R0, #60H
MOV A, @R0
INC R0
L1: MOV B, A
MOV A, @R0
CJNE A, 0F0H, L2
SJMP L3
L2: JC L3
MOV A, B
L3: INC R0
DJNZ R2, L1
MOV 70H, A
HERE: SJMP HERE
END
```

Experiment 4 — 8051 C programming for timer and UART

4a. Timer Mode 1 with delay

```
#include <REG51.H>

sbit LED = P1^0;

void delay(void) {
    TMOD = 0x01;
    TH0 = 0xFC;
    TL0 = 0x74;
    TF0 = 0;
    TR0 = 1;
    while(TF0 == 0);
    TR0 = 0;
    TF0 = 0;
}

void main(void) {
    while(1) {
        LED = !LED;
        delay();
    }
}
```

4b. Timer interrupt to toggle an output pin

```
#include <REG51.H>

sbit LED = P1^0;

void timer0(void) interrupt 1 {
    LED = !LED;
    TH0 = 0x4C;
    TL0 = 0x00;
}

void main(void) {
    TMOD = 0x01;
    TH0 = 0x4C;
    TL0 = 0x00;
    ET0 = 1;
    EA = 1;
    TR0 = 1;

    while(1);
}
```

4c. UART transmission

```
#include <REG51.H>

void main(void) {
    TMOD = 0x20;
    TH1 = 0xFA;
    SCON = 0x50;
    TR1 = 1;

    while(1) {
        SBUF = 'B';
        while(TI == 0);
        TI = 0;
    }
}
```

Experiment 5 — 8051 C programming for port handling

5a. Display an ASCII number in a port

```
#include <REG51.H>

void main(void) {
    while(1) {
        P1 = 0x32;
        P1 = 0x39;
    }
}
```

5b. I/O port programming with delay

```
#include <REG51.H>

sbit LED = P1^7;

void delay(void) {
    unsigned int i, j;
    for(i = 0; i < 20; i++)
        for(j = 0; j < 50; j++);
}

void main(void) {
    while(1) {
        LED = 0;
        delay();
        LED = 1;
        delay();
    }
}
```

5c. Toggle between multiple ports

```
#include <REG51.H>

void delay(void) {
    unsigned int i;
    for(i = 0; i < 500; i++);
}

void main(void) {
    P0 = P1 = P2 = P3 = 0x33;

    while(1) {
        P0 = ~P0;
        P1 = ~P1;
        P2 = ~P2;
        P3 = ~P3;
        delay();
    }
}
```

Experiment 6 — LEDs and switches (AT89C52)

exp6a.c

```
#include <REGX52.H>

#define LED1 P1_5
#define LED2 P1_4
#define LED3 P1_3

void delay(unsigned int count)
{
    unsigned int x, y;
    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}

void main()
{
    while(1)
    {
        LED1=0;
        LED2=1;
        LED3=0;
        delay(50);

        LED1=1;
        LED2=0;
        LED3=0;
        delay(50);
    }
}
```

exp6b.c

```
#include <REGX52.H>

#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2

#define BEEP P1_6
#define LED1 P1_5
#define LED2 P1_4
#define LED3 P1_3
```

```c
void delay(unsigned int count)
{
    unsigned int x, y;
    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}

void main()
{
    SW1=1;
    SW2=1;
    SW3=1;
    BEEP=1;

    LED1=1;
    LED2=1;
    LED3=1;

    while(1)
    {
        if(SW1==0)
        {
            LED1=0; LED2=1; LED3=1;
            BEEP=0; delay(25); BEEP=1;
            while(SW1==0);
        }

        if(SW2==0)
        {
            LED1=1; LED2=0; LED3=1;
            BEEP=0; delay(25); BEEP=1;
            while(SW2==0);
        }

        if(SW3==0)
        {
            LED1=1; LED2=1; LED3=0;
            BEEP=0; delay(25); BEEP=1;
            while(SW3==0);
        }
    }
}
```

Experiment 7 — LCD interfacing (AT89C52)

exp7a.c

```c
#include <REGX52.H>

#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1

void delay(unsigned int);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, char *);
void lcd_initial(void);

void main()
{
    lcd_initial();

    lcd_display(0x80, "AJAY.I");
    lcd_display(0xC0, "3rd Year");

    while(1);
}

void lcd_initial(void)
{
    delay(20);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
}

void lcd_command(char value)
{
    LCD_PORT = value;
    RS = 0;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_data(char value)
{
    LCD_PORT = value;
    RS = 1;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_display(unsigned char locn, char *dat)
{
    lcd_command(locn);

    while(*dat)
    {
        lcd_data(*dat);
        dat++;
    }
}

void delay(unsigned int count)
{
    unsigned int x, y;

    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}
```

exp7b.c

```c
#include <REGX52.H>

#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1

#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2
#define BEEP P1_6

void delay(unsigned int);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, char *);
void lcd_initial(void);

void main()
{
    P2 = 0x07;
    BEEP = 1;

    lcd_initial();
    lcd_display(0x80, "Press Any Switch");

    while(1)
    {
        if(SW1 == 0)
        {
            while(SW1 == 0);
            BEEP = 0;
            delay(25);
            BEEP = 1;
            lcd_command(0x01);
            lcd_display(0x80, "Pressed Switch 1");
        }

        if(SW2 == 0)
        {
            while(SW2 == 0);
            BEEP = 0;
            delay(25);
            BEEP = 1;
            lcd_command(0x01);
            lcd_display(0x80, "Pressed Switch 2");
        }

        if(SW3 == 0)
        {
            while(SW3 == 0);
            BEEP = 0;
            delay(25);
            BEEP = 1;
            lcd_command(0x01);
            lcd_display(0x80, "Pressed Switch 3");
        }
    }
}

void lcd_initial(void)
{
    delay(20);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
}

void lcd_command(char value)
{
    LCD_PORT = value;
    RS = 0;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}
```

```c
void lcd_data(char value)
{
    LCD_PORT = value;
    RS = 1;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_display(unsigned char locn, char *dat)
{
    lcd_command(locn);

    while(*dat)
    {
        lcd_data(*dat);
        dat++;
    }
}

void delay(unsigned int count)
{
    unsigned int x, y;

    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}
```

Experiments 8, 9 and 10 — ARM7 (LPC21xx)

exp8.c

```c
#include <lpc21xx.h>

int main()
{
    IO0DIR = 0x00000080;

    while(1)
    {
        IO0SET = 0x00000080;
        for(int i=0; i<500000; i++);

        IO0CLR = 0x00000080;
        for(int i=0; i<500000; i++);
    }
}
```

exp9.c

```c
#include <lpc21xx.h>

void delay(unsigned int n)
{
    unsigned int i;
    for(i=0; i<n; i++);
}

int main(void)
{
    IO0DIR |= 0x00010000;

    while(1)
    {
        IO0SET = 0x00010000;
        delay(500000);

        IO0CLR = 0x00010000;
        delay(500000);
    }
}
```

exp10.c

```c
#include <lpc21xx.h>

void timer0_delay_ms(unsigned int ms)
{
    T0TCR = 0x02;
    T0PR = 14999;
    T0MR0 = ms;
    T0MCR = 0x04;
    T0TC = 0;
    T0TCR = 0x01;

    while(T0TC < ms);

    T0TCR = 0x00;
}

int main(void)
{
    IO0DIR |= 0x00000080;

    while(1)
    {
        IO0SET = 0x00000080;
        timer0_delay_ms(500);

        IO0CLR = 0x00000080;
        timer0_delay_ms(500);
    }
}
```